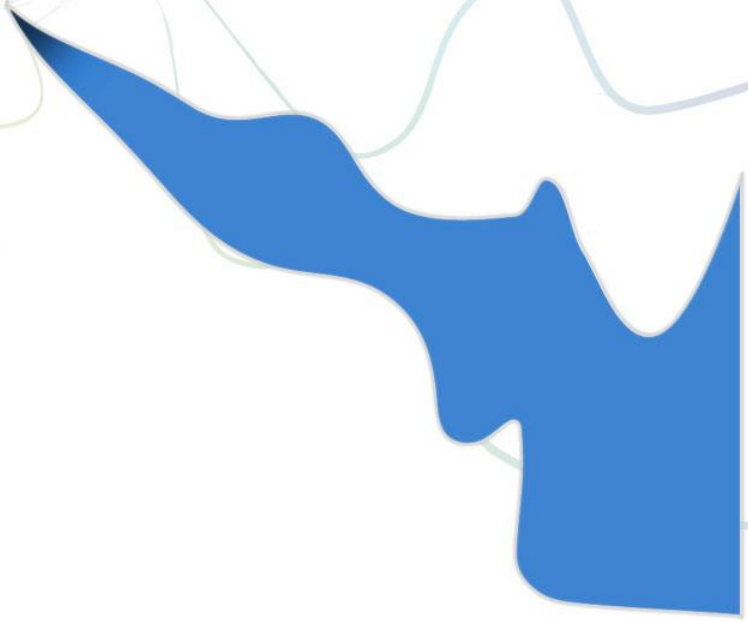


Théorie des graphes

Chapitre 3
Coloriage, parcours des graphes et plus courts chemins

Coloriage de graphes

Le problème du coloriage de graphes, pour un graphe non orienté G, consiste à attribuer une couleur à chaque sommet, de telle sorte qu'une même couleur ne soit pas attribuée à deux sommets adjacents (reliés par une arête).

Coloriage de graphes

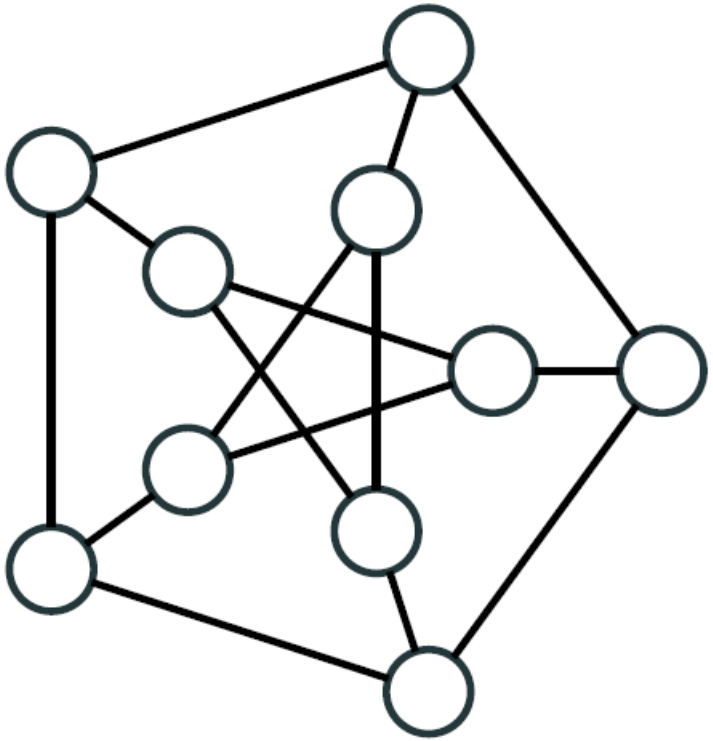
Le problème du coloriage de graphes, pour un graphe non orienté G, consiste à attribuer une couleur à chaque sommet, de telle sorte qu'une même couleur ne soit pas attribuée à deux sommets adjacents (reliés par une arête).

Définition

Le nombre minimum de couleurs nécessaires pour colorier un graphe G est appelé le nombre chromatique de G, et noté X(G).

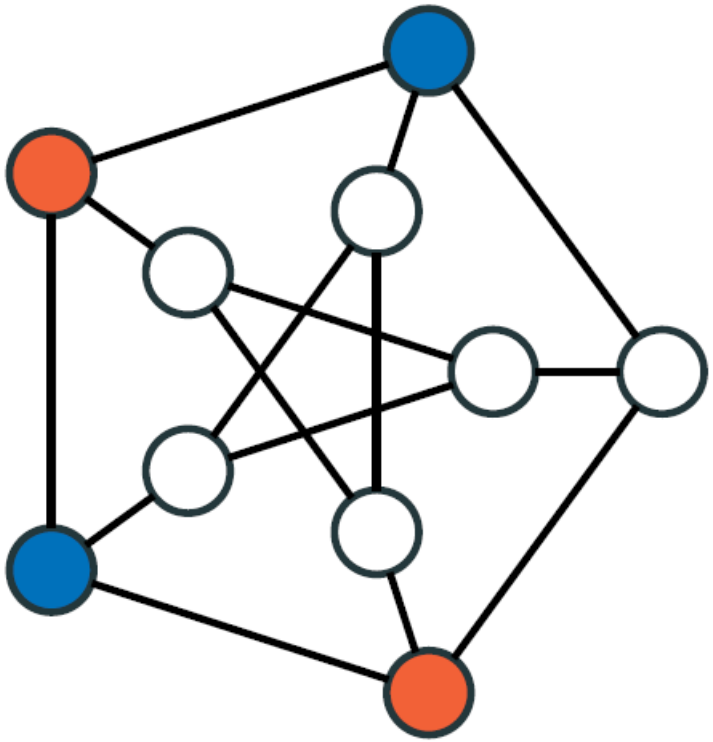
Coloriage de graphes

- Exemple:



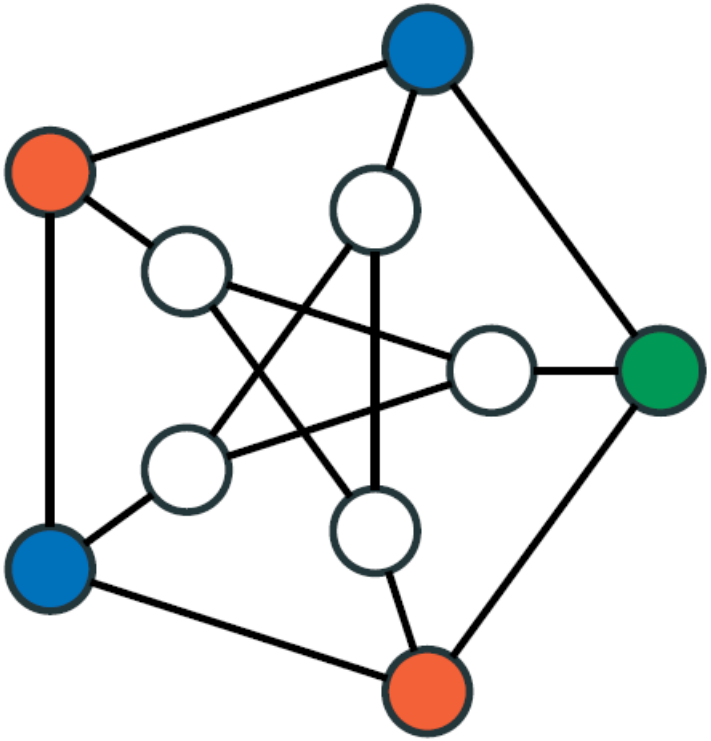
Coloriage de graphes

- Exemple:



Coloriage de graphes

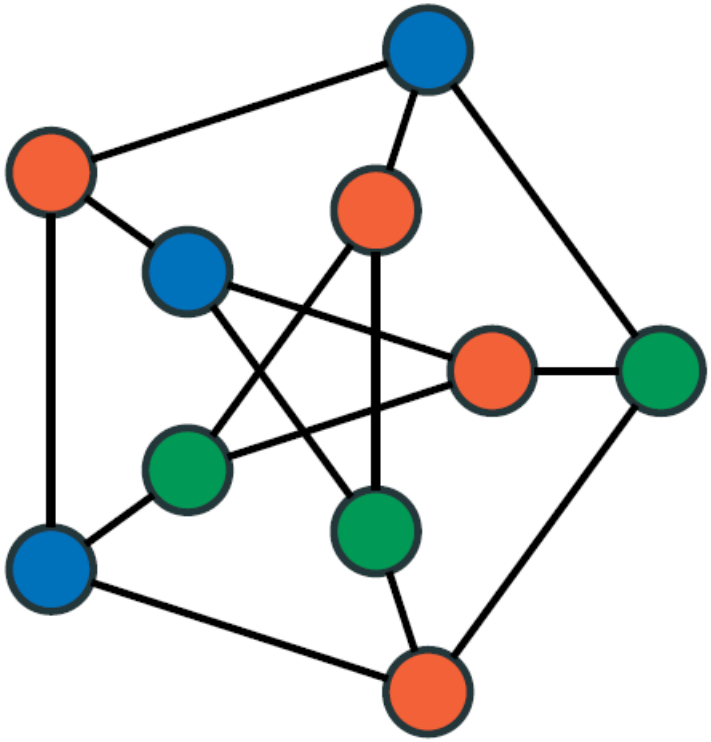
- Exemple:



Coloriage de graphes

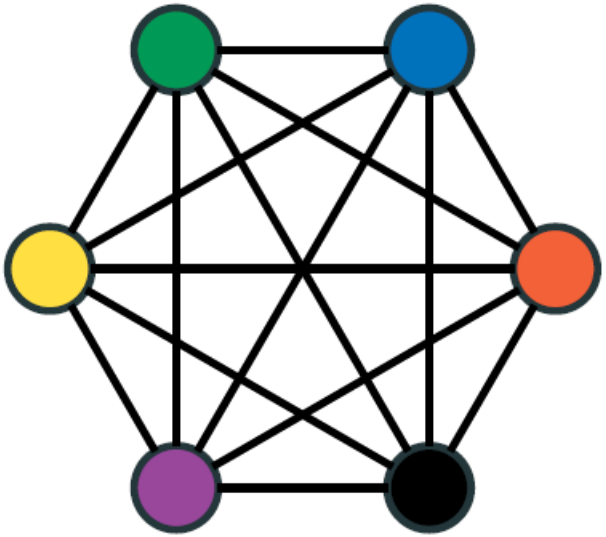
- Exemple:

Le nombre chromatique de G est $X(G)=3$



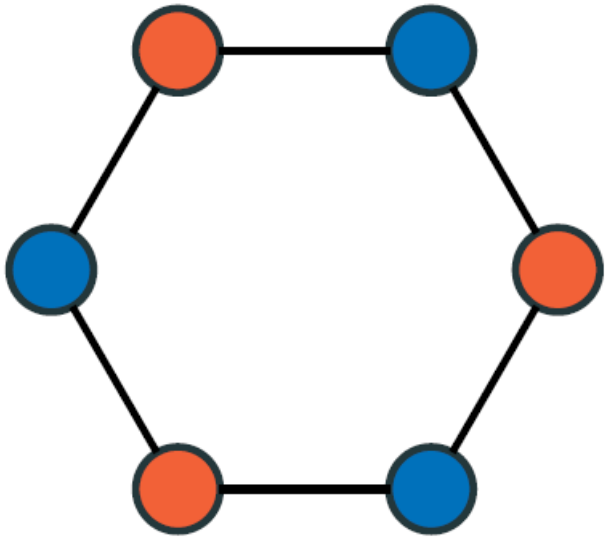
Coloriage de graphes

Le nombre chromatique de K_n est n



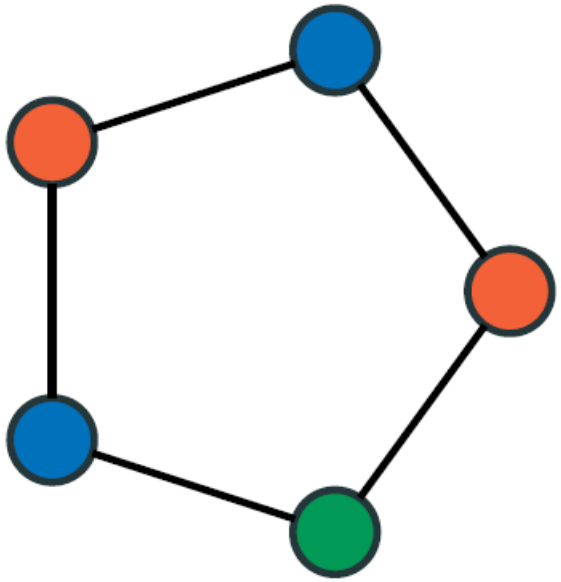
Coloriage de graphes

Pour n pair, Le nombre chromatique de C_n (cycle) est 2



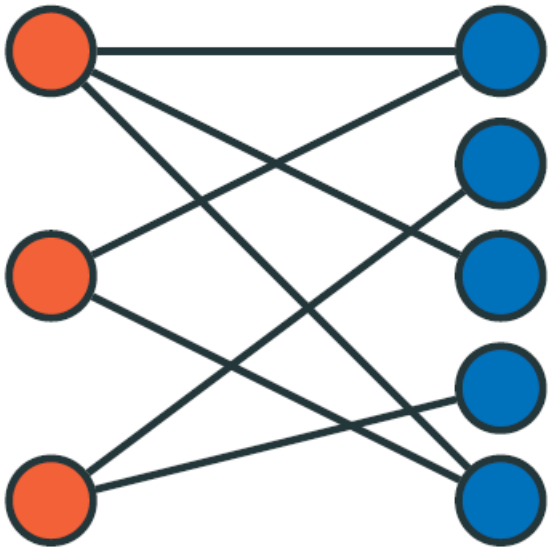
Coloriage de graphes

Pour n>2 impair, Le nombre chromatique de C_n est 3



Coloriage de graphes

Le nombre chromatique d'un graphe biparti est 2



Coloriage de graphes

Etant donné un graphe non orienté G = (S, A),

Définition

- Un sous-ensemble S de V est un stable s'il ne comprend que des sommets non adjacents deux à deux.
- Le cardinal du plus grand stable est le nombre de stabilité de G ; on le note $\alpha(G)$.
- une clique est un sous-ensemble de sommets $S' \subseteq S$ qui sont tous connectés 2 à 2.

Coloriage de graphes

- Encadrement du nombre chromatique

Le nombre chromatique d'un graphe G vérifie

$$X(G) <= r+1$$

où *r* est le plus grand degré des sommets de *G*.

$$X(G) \leq n+1-\alpha(G)$$

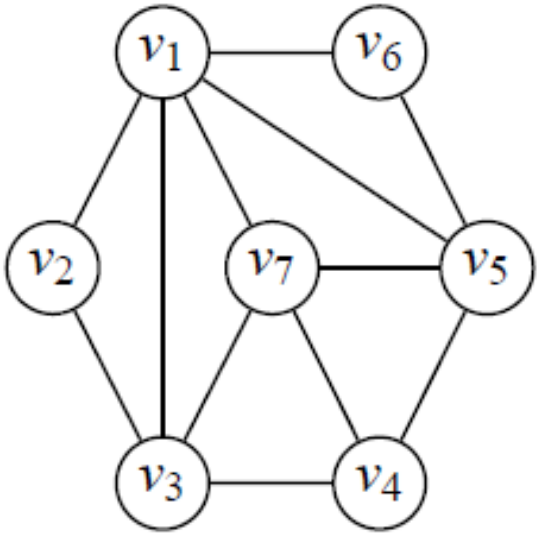
où $\alpha(G)$ est le nombre de stabilité de G.

$$X(G) \geq w(G)$$

où $w(G)$ est l'ordre de sa plus grande clique

Coloriage de graphes

- Exercice: Majorez et minorez le nombre chromatique de ce graphe.



- Correction…

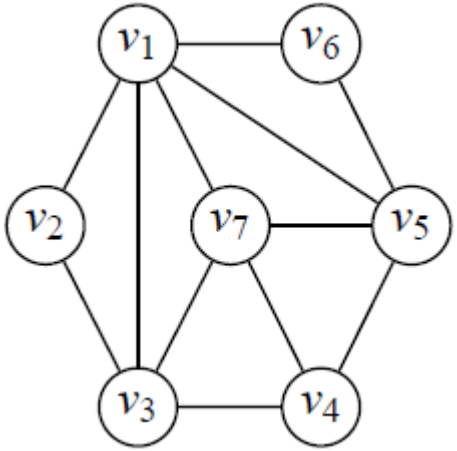
Coloriage de graphes

Algorithme de coloration de Welsh et Powell

- **Étape 1:** Classer les sommets du graphe dans l'ordre décroissant de leur degré, et attribuer à chacun des sommets son numéro d'ordre dans la liste obtenue.
- **Étape 2:** En parcourant la liste dans l'ordre, attribuer une couleur non encore utilisée au premier sommet non encore coloré, et attribuer cette même couleur à chaque sommet non encore coloré et non adjacent à un sommet de cette couleur.
- **Étape 3:** S'il reste des sommets non colorés dans le graphe, revenir à l'étape 2. Sinon, FIN.

Coloriage de graphes

- Exercice: Utilisez l'algorithme de coloration de Welsh et Powell pour colorer le graphe suivant:



Coloriage de graphes

Conjecture des 4 couleurs

Le nombre chromatique d'un graphe planaire est inférieur ou égal à 4

Une application de cette conjecture est que l'on peut colorier les pays d'une carte géographique avec seulement 4 couleurs de telle sorte que deux pays voisins soient coloriés avec des couleurs différentes.

Parcours de graphes

- Le parcours en largeur consiste à explorer les sommets du graphe niveau par niveau, à partir d'un sommet donné ;

Parcours de graphes

- Le parcours en largeur consiste à explorer les sommets du graphe niveau par niveau, à partir d'un sommet donné ;

- le parcours en profondeur consiste, à partir d'un sommet donné, à suivre un chemin le plus loin possible (jusqu'à un cul-de-sac ou un cycle), puis à faire des retours en arrière pour reprendre tous les chemins ignorés précédemment.

Parcours de graphes

- Dans les deux cas, l'algorithme procède par coloriage des sommets :

Parcours de graphes

- Dans les deux cas, l'algorithme procède par coloriage des sommets :

  1. Initialement, tous les sommets sont blancs. On dira qu'un sommet blanc n'a pas encore été découvert.

Parcours de graphes

- Dans les deux cas, l'algorithme procède par coloriage des sommets :

1. Initialement, tous les sommets sont blancs. On dira qu'un sommet blanc n'a pas encore été découvert.
2. Lorsqu'un sommet est "découvert" , il est colorié en gris.

Parcours de graphes

- Dans les deux cas, l'algorithme procède par coloriage des sommets :

1. Initialement, tous les sommets sont blancs. On dira qu'un sommet blanc n'a pas encore été découvert.
2. Lorsqu'un sommet est "découvert" , il est colorié en gris.
3. Un sommet est colorié en noir lorsque tous ses successeurs sont gris ou noirs

Parcours de graphes

- De façon pratique, on va utiliser une liste “d’attente au coloriage en noir” dans laquelle on va stocker tous les sommets gris.

Parcours de graphes

- De façon pratique, on va utiliser une liste “d’attente au coloriage en noir” dans laquelle on va stocker tous les sommets gris.
- La différence fondamentale entre le parcours en largeur et le parcours en profondeur provient de la façon de gérer cette liste d’attente au coloriage en noir :

Parcours de graphes

- De façon pratique, on va utiliser une liste “d’attente au coloriage en noir” dans laquelle on va stocker tous les sommets gris.
- La différence fondamentale entre le parcours en largeur et le parcours en profondeur provient de la façon de gérer cette liste d’attente au coloriage en noir :
  1. Le parcours en largeur utilise une file d’attente, où le premier sommet arrivé dans la file est aussi le premier à en sortir

Parcours de graphes

- De façon pratique, on va utiliser une liste "d'attente au coloriage en noir" dans laquelle on va stocker tous les sommets gris.
- La différence fondamentale entre le parcours en largeur et le parcours en profondeur provient de la façon de gérer cette liste d'attente au coloriage en noir :
  1. Le parcours en largeur utilise une file d'attente, où le premier sommet arrivé dans la file est aussi le premier à en sortir
  2. Le parcours en profondeur utilise une pile, où le dernier sommet arrivé dans la pile est le premier à en sortir.

Parcours de graphes

❑ Parcours en largeur (Breadth First Search = BFS)

- Structures de données utilisées :

Parcours de graphes

❑ Parcours en largeur (Breadth First Search = BFS)

- Structures de données utilisées :
  1. On utilise une file F :
     - ✓ init_file(F) qui initialise la file F à vide,
     - ✓ enfiler(F,s) qui ajoute le sommet s à la fin de la file F,
     - ✓ est_vide(F) qui retourne vrai si la file F est vide et faux sinon,
     - ✓ defiler(F,s) qui enlève le sommet s au début de la file F.

Parcours de graphes

❑ Parcours en largeur (Breadth First Search = BFS)

- Structures de données utilisées :
  1. On utilise une file F :
     - ✓ init_file(F) qui initialise la file F à vide,
     - ✓ enfiler(F,s) qui ajoute le sommet s à la fin de la file F,
     - ✓ est_vide(F) qui retourne vrai si la file F est vide et faux sinon,
     - ✓ defiler(F,s) qui enlève le sommet s au début de la file F.
  2. On utilise un tableau π qui associe à chaque sommet le sommet qui l'a fait entrer dans la file et un tableau *couleur* qui associe à chaque sommet sa couleur (blanc, gris ou noir).

Parcours de graphes

❑ Parcours en largeur (Breadth First Search = BFS)

- Structures de données utilisées :
  1. On utilise une file F :
     - ✓ init_file(F) qui initialise la file F à vide,
     - ✓ enfiler(F,s) qui ajoute le sommet s à la fin de la file F,
     - ✓ est_vide(F) qui retourne vrai si la file F est vide et faux sinon,
     - ✓ defiler(F,s) qui enlève le sommet s au début de la file F.
  2. On utilise un tableau π qui associe à chaque sommet le sommet qui l'a fait entrer dans la file et un tableau *couleur* qui associe à chaque sommet sa couleur (blanc, gris ou noir).
  3. On va utiliser de plus un tableau d qui associe à chaque sommet son niveau de profondeur par rapport au sommet de départ s_0

Parcours de graphes

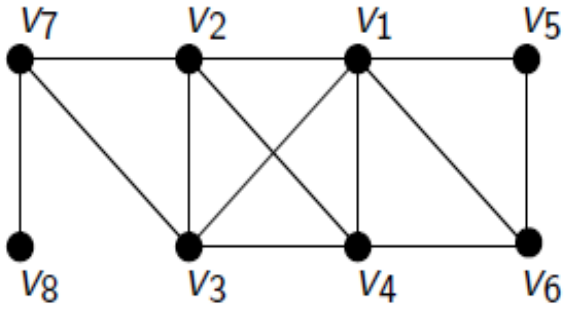
Algorithme de parcours en largeur (BFS) (S; A; s_0)

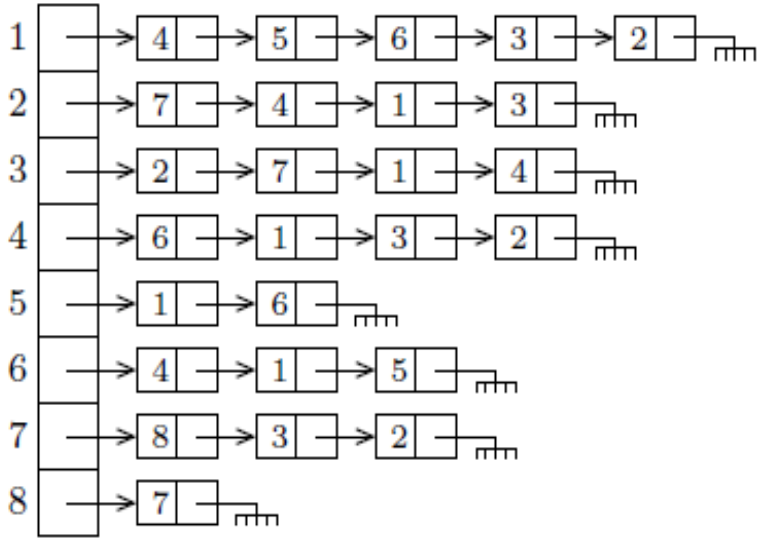
init_file(F)
pour tout sommet s_i ϵ S faire
$\pi[s_i] \leftarrow$ Nil;
$d[s_i] \leftarrow \infty$;
couleur$[s_i] \leftarrow$ blanc;
fin pour
$d[s_0] \leftarrow 0$;
enfiler(F; s_0);
couleur$[s_0] \leftarrow$ gris;
tant que est_vide(F) = faux faire
defiler(F; s_i);
pour tout s_j ϵ succ(s_i) faire
si couleur$[s_j]$ = blanc alors
enfiler(F; s_j);
couleur$[s_j] \leftarrow$ gris;
$\pi[s_j] \leftarrow s_i$;
$d[s_j] \leftarrow d[s_i] + 1$;
finsi
fin pour
couleur$[s_i] \leftarrow$ noir;
fin tant
fin BFS

Parcours de graphes

Exemple :

Soit un graphe G et une représentation par listes d'adjacence.





Parcours en largeur à partir de la racine 7.

Parcours de graphes

- Applications du parcours en largeur
  - ✓ Rechercher l'ensemble des sommets accessibles depuis s_0.

Parcours de graphes

- Applications du parcours en largeur
  - ✓ Rechercher l'ensemble des sommets accessibles depuis s_0.
  - ✓ Déterminer les composantes connexes d'un graphe non orienté.

Parcours de graphes

Applications du parcours en largeur

- ✓ Rechercher l'ensemble des sommets accessibles depuis s_0.
- ✓ Déterminer les composantes connexes d'un graphe non orienté.
- ✓ Chercher le plus court chemin (en nombre d'arcs ou arêtes) entre la racine s_0 et chacun des autres sommets du graphe accessibles depuis s_0.

Parcours de graphes

Applications du parcours en largeur

- ✓ Rechercher l'ensemble des sommets accessibles depuis s_0.
- ✓ Déterminer les composantes connexes d'un graphe non orienté.
- ✓ Chercher le plus court chemin (en nombre d'arcs ou arêtes) entre la racine s_0 et chacun des autres sommets du graphe accessibles depuis s_0.
- ✓ Tester si un graphe est biparti

Parcours de graphes

Applications du parcours en largeur

- ✓ Rechercher l'ensemble des sommets accessibles depuis s_0.
- ✓ Déterminer les composantes connexes d'un graphe non orienté.
- ✓ Chercher le plus court chemin (en nombre d'arcs ou arêtes) entre la racine s_0 et chacun des autres sommets du graphe accessibles depuis s_0.
- ✓ Tester si un graphe est biparti
- ✓ …

Parcours de graphes

❑ Parcours en profondeur (Depth First Search = DFS)

- Structures de données utilisées :

Parcours de graphes

❑ Parcours en profondeur (Depth First Search = DFS)

- Structures de données utilisées :

1. On utilise une pile P, pour laquelle on suppose définies les fonctions
   - ✓ init_pile(P) qui initialise la pile P à vide,
   - ✓ empile(P,s) qui ajoute s au sommet de la pile P,
   - ✓ est_vide(P) qui retourne vrai si la pile P est vide et faux sinon,
   - ✓ sommet(P) qui retourne le sommet s au sommet de la pile P,
   - ✓ depile(P,s) qui enlève s du sommet de la pile P.

Parcours de graphes

❑ Parcours en profondeur (Depth First Search = DFS)

- Structures de données utilisées :

1. On utilise une pile P, pour laquelle on suppose définies les fonctions
   - ✓ init_pile(P) qui initialise la pile P à vide,
   - ✓ empile(P,s) qui ajoute s au sommet de la pile P,
   - ✓ est_vide(P) qui retourne vrai si la pile P est vide et faux sinon,
   - ✓ sommet(P) qui retourne le sommet s au sommet de la pile P,
   - ✓ depile(P,s) qui enlève s du sommet de la pile P.
2. On utilise, comme pour le parcours en largeur, un tableau π qui associe à chaque sommet le sommet qui l'a fait entrer dans la pile, et un tableau *couleur* qui associe à chaque sommet sa couleur (blanc, gris ou noir).

Parcours de graphes

Algorithme de parcours en profondeur (DFS) (S; A; s_0)

init_pile(P)

pour tout sommet s_i ϵ S faire

$\pi[s_i] \leftarrow$ Nil;

couleur$[s_i] \leftarrow$ blanc;

fin pour

empile(P; s_0)

couleur$[s_0] \leftarrow$ gris;

tant que est_vide(P) = faux faire

$s_i \leftarrow$ sommet(P)

si ∃ s_j ϵ succ(s_i) tel que couleur$[s_j]$=blanc alors

empile(P; s_j);

couleur$[s_j] \leftarrow$ gris;

$\pi[s_j] \leftarrow s_i$;

sinon

depile(P; s_i);

couleur$[s_i] \leftarrow$ noir;

finsi

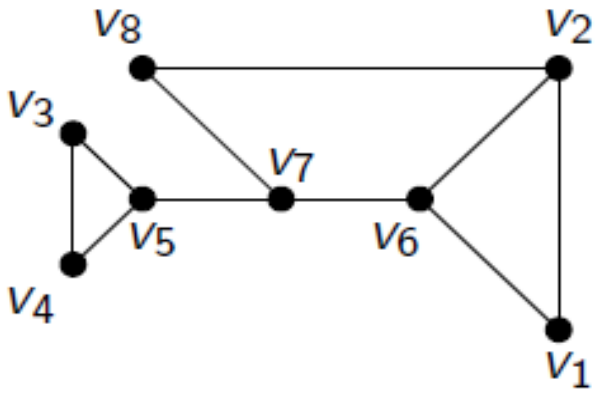
fin tant

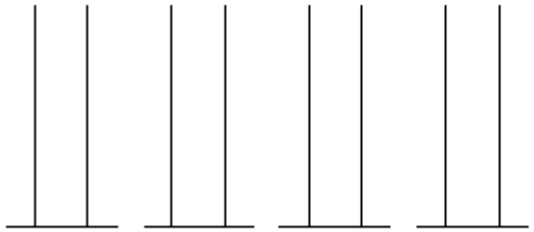
fin DFS

Parcours de graphes

- Exemple :

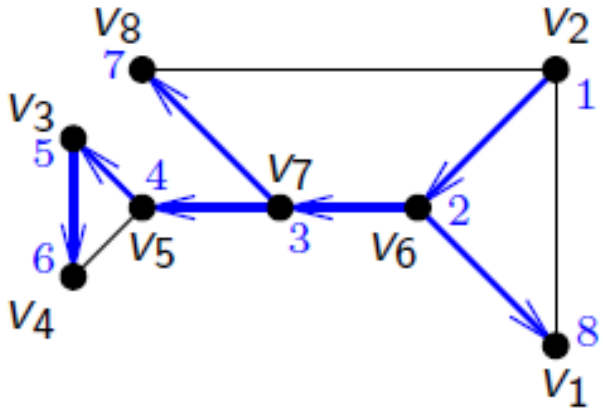
LS	2	6	6	1	8	4	5	3	5	3	4	7	2	7	1	6	5	8	2	7

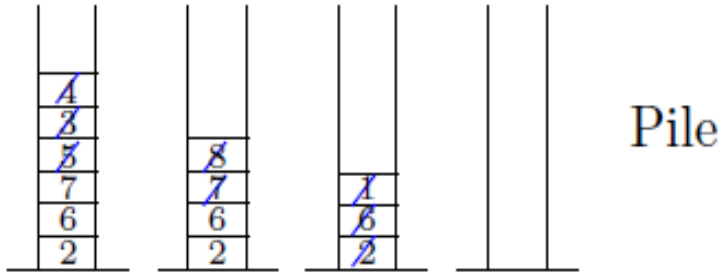




Parcours de graphes

- Exemple :





Parcours de graphes

- Applications du parcours en profondeur

  - ✓ Recherche des composantes connexes

Parcours de graphes

- Applications du parcours en profondeur

- ✓ Recherche des composantes connexes
- ✓ Recherche de circuits

Parcours de graphes

- Applications du parcours en profondeur

  - ✓ Recherche des composantes connexes
  - ✓ Recherche de circuits
  - ✓ Recherche des composantes fortement connexes d'un graphe orienté

Parcours de graphes

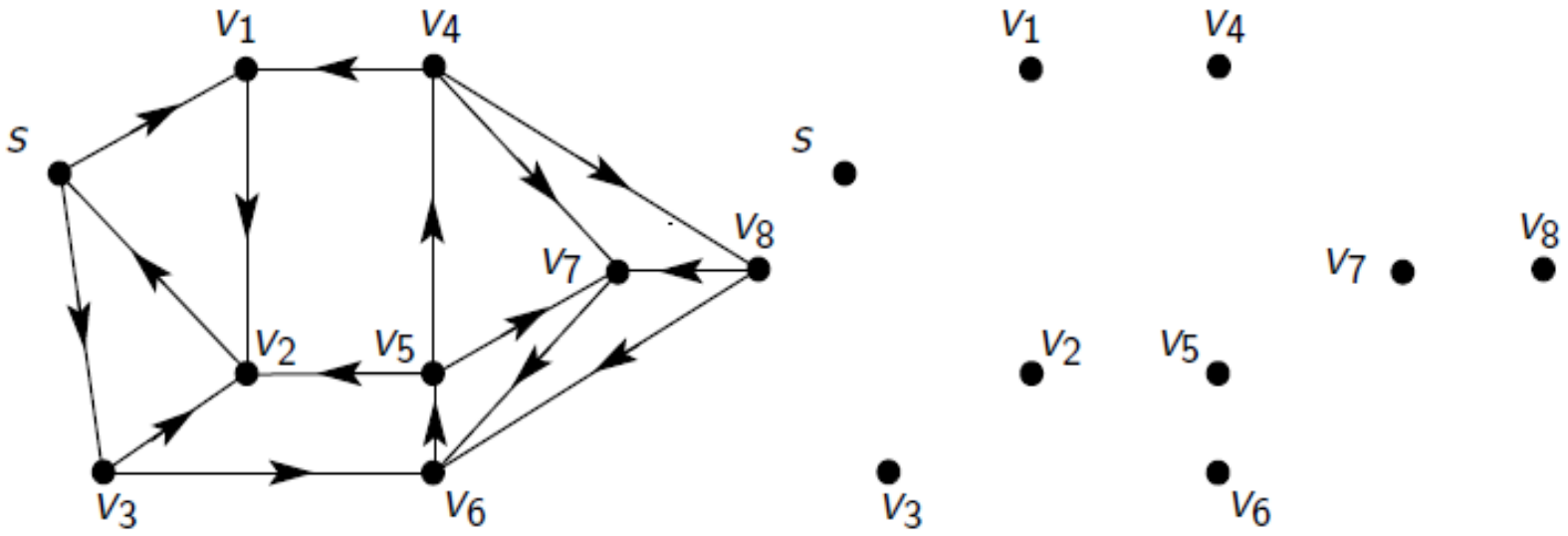
- Applications du parcours en profondeur

  - ✓ Recherche des composantes connexes
  - ✓ Recherche de circuits
  - ✓ Recherche des composantes fortement connexes d'un graphe orienté
  - ✓ …

Parcours de graphes

Exemple :

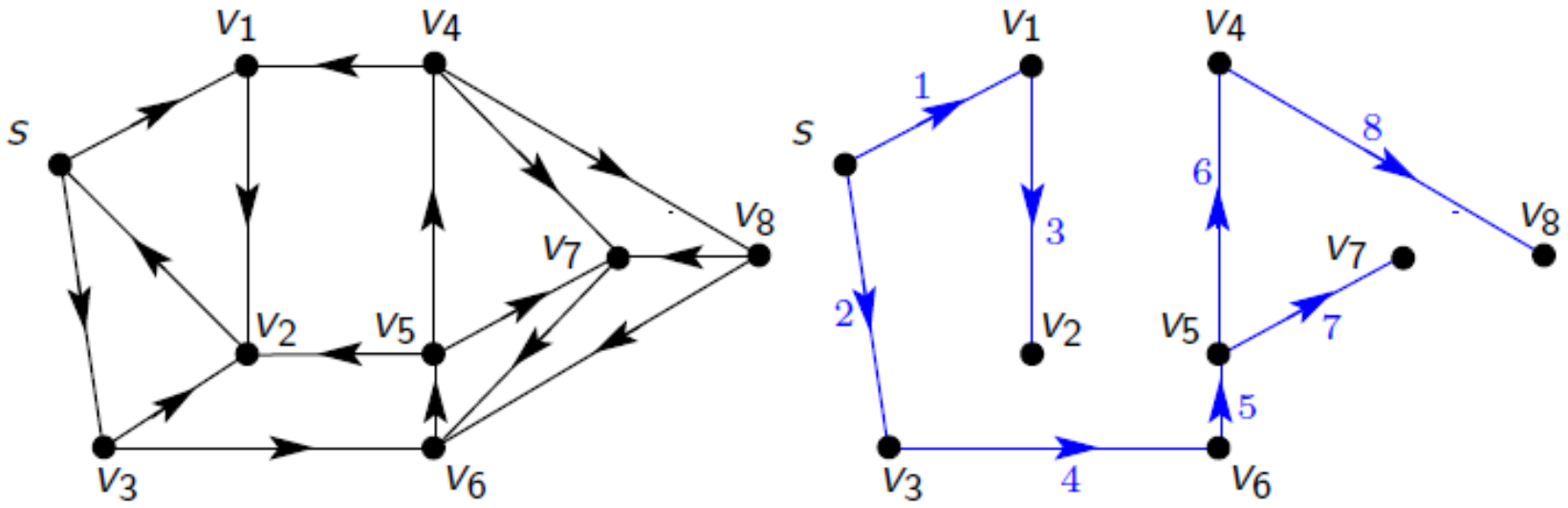
Parcours en largeur à partir de s



Parcours de graphes

Exemple :

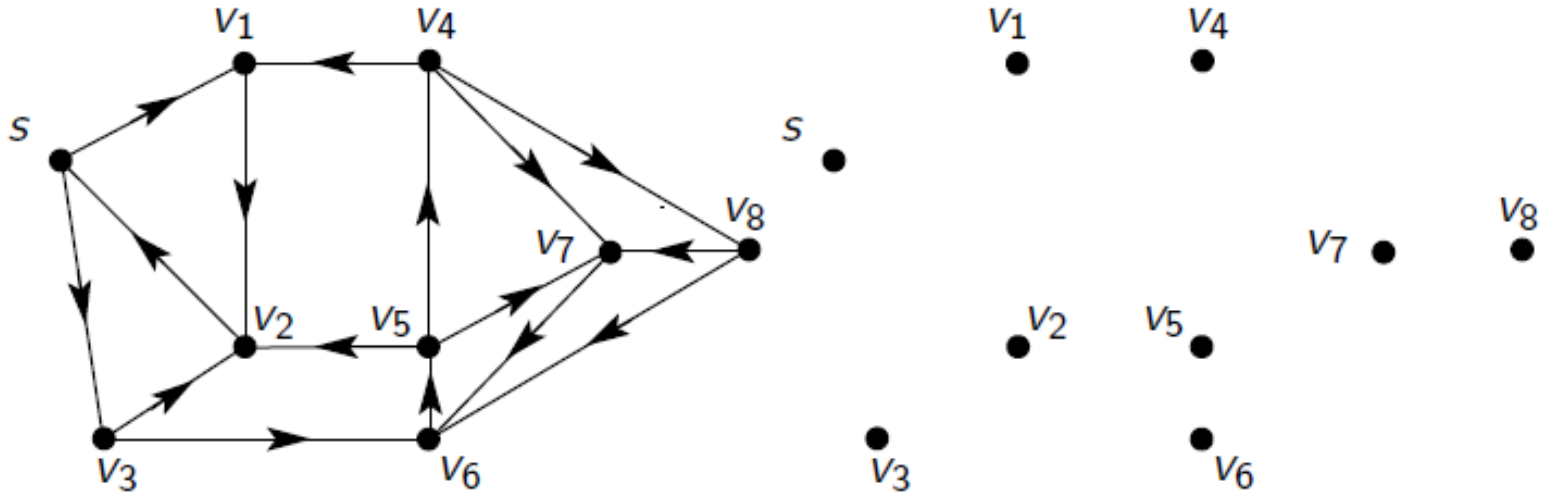
Parcours en largeur à partir de s



Parcours de graphes

Exemple :

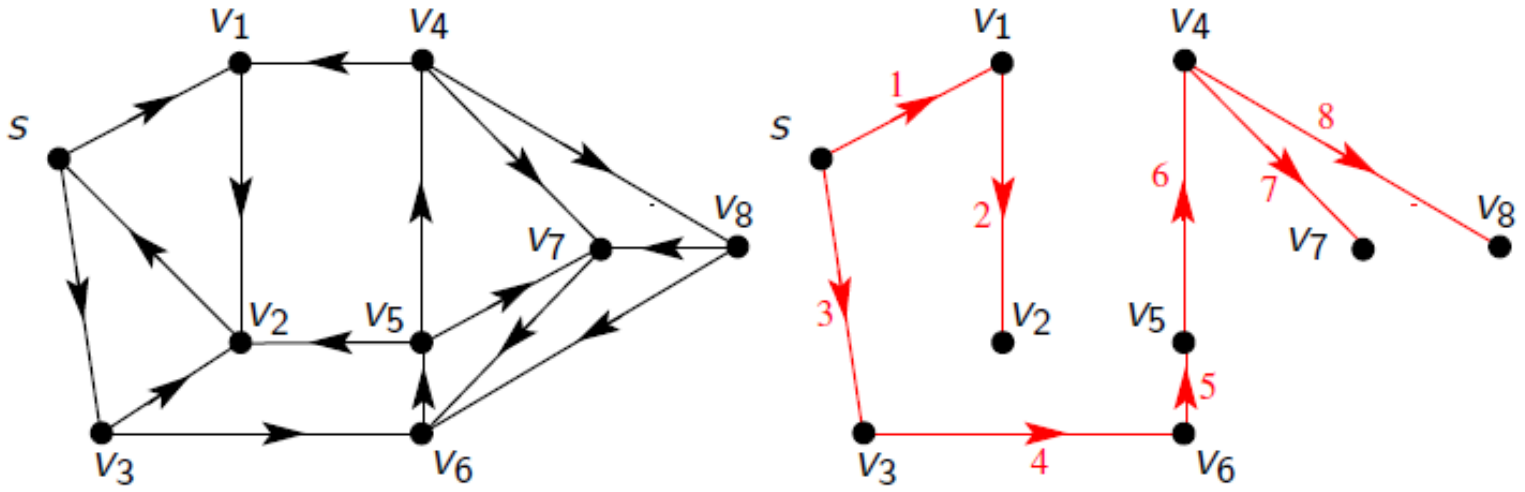
Parcours en profondeur à partir de s



Parcours de graphes

Exemple :

Parcours en profondeur à partir de s



Plus courts chemins

- Définitions
- Soit G = (S;A) un 1-graphe orienté valué tel que à chaque arc (s_i; s_j) de A un coût réel est associé cout(s_i; s_j).
- Le coût d'un chemin p =< s_0; s_1; s_2;…. s_k> est égal à la somme des coûts des arcs composant le chemin, c'est à dire,

$$\text{Cout(p)} = \sum_{n=1}^{k} cout(s_{i-1}, s_i)$$

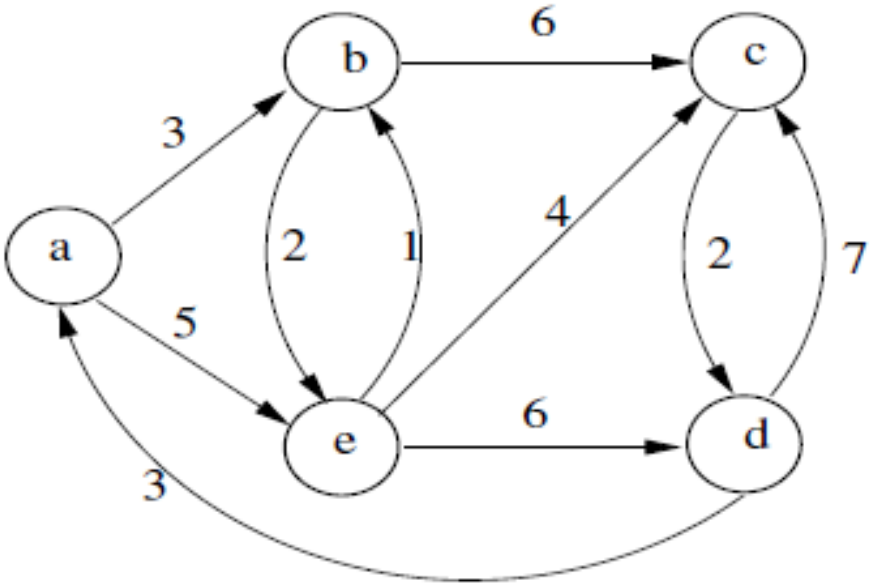
- Le coût (ou poid) d'un plus court chemin entre deux sommets s_i et s_j est noté δ(si; sj) et est défini par

δ(si; sj)=+∞ S'il n y a pas de chemin entre s_i et s_j

δ(si; sj)=min{cout(p)/p chemin de s_i et s_j} Sinon

Plus courts chemins

- Considérons par exemple le graphe valué orienté suivant :



- on a $\delta(a; b) = 3$, $\delta(a; e) = 5$, $\delta(a; c) = 9$ et $\delta(a; d) = 11$.

Plus courts chemins

- Problème des plus courts chemins à origine unique

Etant donné un graphe orienté G = (S;A), une fonction cout : A → R et un sommet origine s_0 ϵ S, on souhaite calculer pour chaque sommet s_j ϵ S le coût (s_0; s_j) du plus court chemin de s_0 à s_j . On supposera que le graphe G ne comporte pas de circuit absorbant (circuit de cout négatif).

- ✓ l'algorithme de Dijkstra résout ce problème lorsque tous les coûts sont positifs ou nuls,
- ✓ l'algorithme de Ford-Bellman résout ce problème lorsque les coûts sont positifs, nuls ou négatifs, sous réserve qu'il n'y ait pas de circuit absorbant (de coût négatif).

Plus courts chemins

- **Algorithme de Dijkstra**
- Supposons que l'on s'intéresse aux chemins partant d'un sommet s_0 .
- On construit un vecteur $d=(d(s_0);d(s_1); \ldots ;d(s_n))$ ayant $n+1$ composantes tel que $d(s_j)$ soit égal à la longueur du plus court chemin allant de s_0 au sommet s_j.
- On initialise ce vecteur comme suit:

$$\begin{cases} d(s_0)=0 & \\ d(s_j)= coup(s_i,s_j) & \text{Si } (s_i, s_j) \in E \\ d(s_j)= \infty & \text{Sinon} \end{cases}$$

où $coup(s_i,s_j) > 0$ est le coup/poids de l'arc (s_i, s_j).

Plus courts chemins

- **Algorithme de Dijkstra**
- On construit un autre vecteur π pour mémoriser le chemin pour aller du sommet s_0 au sommet voulu. La valeur $\pi(s_i)$ donne le sommet qui précède s_i dans le chemin.
- On initialise ce vecteur à NIL
- On considère ensuite deux ensembles de sommets, S initialisé à $\{s_0\}$ et T initialisé à $\{s_1, s_2, \ldots, s_n\}$.
- À chaque pas de l'algorithme, on ajoute à S un sommet jusqu'à ce que $S = V$ de telle sorte que le vecteur d donne à chaque étape la longueur minimale des chemins de s_0 aux sommets de S.

Plus courts chemins

- **Algorithme de Dijkstra**

Début

Pour tout sommet s_j ϵ T faire

Si $d(s_j) < \infty$ alors $\pi(s_j) = s_0$;

FinPour

Tant que T n'est pas vide faire

Choisir s_i dans T tel que $d(s_i)$ est minimum

Retirer s_i de T et l'ajouter à S

Pour chaque successeur s_j de s_i, avec s_j dans T faire

Si $d(s_j) > d(s_i)+coup(s_i, s_j)$ alors

$d(s_j) = d(s_i)+coup(s_i, s_j)$;

$\pi(s_j) = s_i$;

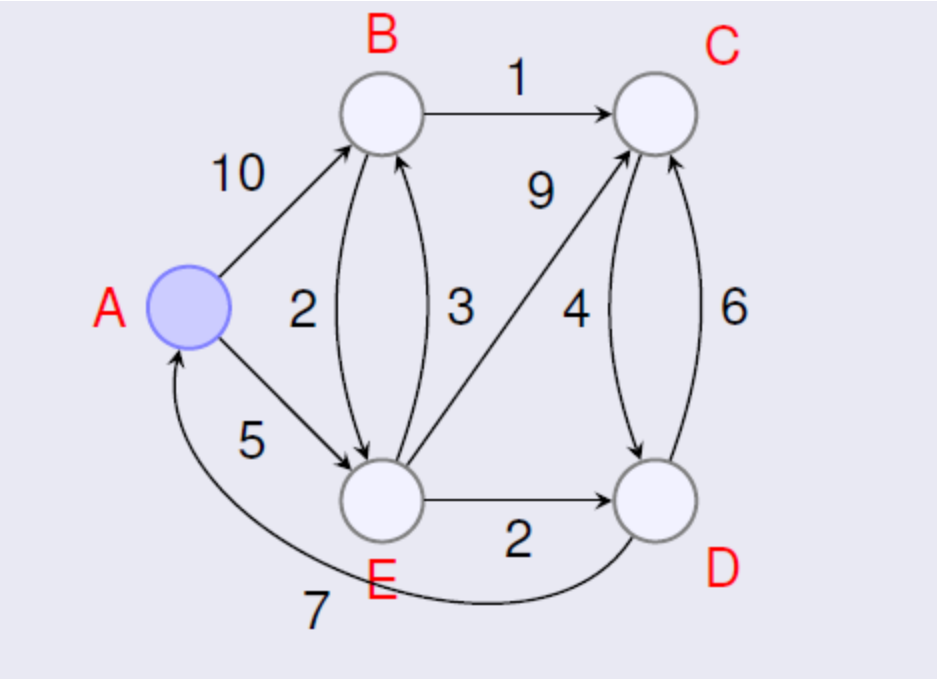
FinPour

FinTantque

Fin

Plus courts chemins

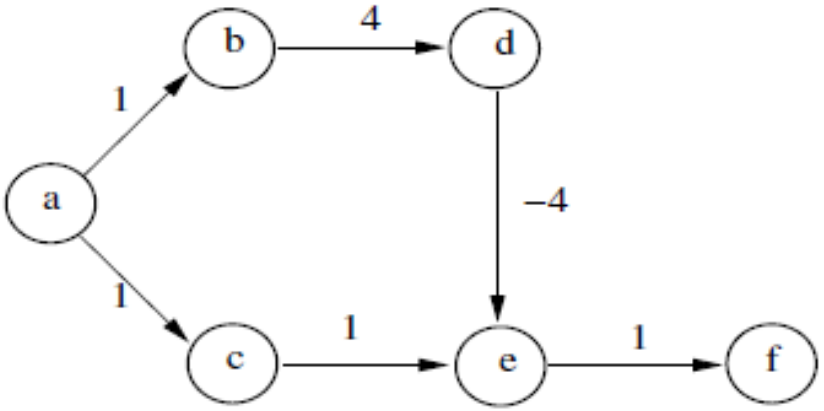
- **Algorithme de Dijkstra:**
- Exemple : Cherchons les plus courts chemins d'origine A dans ce graphe



Plus courts chemins

- **Algorithme de Bellman-Ford**

L'algorithme de Dijkstra ne marche pas toujours quand le graphe contient des arcs dont les coûts sont négatifs. Considérons par exemple le graphe suivant :



Pour aller de a à f, l'algorithme de Dijkstra va trouver le chemin < a; c; e; f >, de poids 3, alors que le chemin < a; b; d; e; f > est de poids 2.

Plus courts chemins

- **Algorithme de Bellman-Ford**

L'algorithme de Bellman-Ford permet de trouver les plus courts chemins à origine unique dans le cas où le graphe contient des arcs dont le coût est négatif, sous réserve que le graphe ne contienne pas de circuit absorbant (dans ce cas, l'agorithme de Bellman-Ford va détecter l'existance de circuits absorbants).

Plus courts chemins

- **Algorithme de Bellman-Ford**

pour tout sommet v ϵ V faire
d[v]= ∞; π[v]=NIL;
d[s] = 0;
pour i de 1 à |V| − 1 faire
pour tout arc (u, v) ϵ E faire // relâchement de l'arc (u, v)
si d[v] > d[u] + w(u, v) alors
d[v] =d[u] + w(u, v); π[v]= u;
pour tout arc (u, v) ϵ E faire // détection des circuits négatifs
si d[v] > d[u] + w(u, v) alors retourner Faux
retourner Vrai

Plus courts chemins

- **Algorithme de Bellman-Ford: Exemple**

On considère le graphe ci-dessous. Faire tourner l'algorithme en prenant comme source le sommet z. Même question en changeant le poids de l'arc (y, v) à 4.

